

R^2 Loss: Range Restriction Loss for Model Compression and Quantization

Anonymous Authors¹

Abstract

Model quantization and compression is widely used techniques to reduce usage of computing resource at inference time. While state-of-the-art works have been achieved reasonable accuracy with higher bit such as 4bit or 8bit, but still it is challenging to quantize/compress a model further, e.g., 1bit or 2bit. To overcome the challenge, we focus on outliers in weights of a pre-trained model which disrupt effective lower bit quantization and compression. In this work, we propose Range Restriction Loss (R^2 loss) for building lower bit quantization and compression friendly models by removing outliers from weights during pre-training. By effectively restricting range of weights, we mold the overall distribution into a tight shape to ensure high quantization bit resolution, therefore allowing model compression and quantization techniques can to utilize their limited numeric representation powers better. We introduce three different, L_∞ R^2 loss, its extension Margin R^2 loss and a new Soft-Min-Max R^2 loss to be used as an auxiliary loss during full-precision model training. These R^2 loss can be used in different cases such as L_∞ and Margin R^2 loss would be effective for symmetric quantization, while Soft-Min-Max R^2 loss shows better performance for model compression. In our experiment, R^2 loss improves lower bit quantization accuracy with state-of-the-art post-training quantization (PTQ), quantization-aware training (QAT), and model compression techniques. With R^2 loss, MobileNet-V2 2bit weight and 8bit activation PTQ, MobileNet-V1 2bit weight and activation QAT, ResNet18 1bit weight compression are improved to 59.49% from 50.66%, 59.05% from 55.96%, and 52.58% from 45.54%, respectively.

¹Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the International Conference on Machine Learning (ICML). Do not distribute.

1. Introduction

Deep neural networks have become popular in human computer interaction, anomaly detection, financial markets, etc. Since a lot of applications running these models run on edge devices, running these compute-intensive models requires a balance of accuracy, latency, power efficiency and size for these models.

Quantization is an effective way of reducing the power, latency, and size of neural networks. This requires that these quantized models are executed on specialized platforms with low-bit supports and at the cost of accuracy drop (Dong et al.; Wang et al., 2019). Post-training quantization involves distributing all the available weights in a layer into bins spread equally apart across the range. Quantization-aware training techniques (Bengio et al., 2013; Zhou et al., 2016), use stochastic gradient descent to quantize and optimize the weights (i.e., mapping each floating point number to a set of discrete bins according to the precision target). Quantization bit-resolution is inversely proportional to the range of weights and affects accuracy of the quantized models. Since outliers tend to increase range, outliers are detrimental for quantization friendly models.

As an example, let's assume we want to quantize the weight distributions shown in Figure 1 (left) into 3 bins. For the original distribution in black most of the weights will be quantized to zero and the model accuracy would drop significantly. This problem gets worse for low bit quantization and compression such as one or two bit quantization.

We introduce **Range Restriction Loss (R^2 loss)**, a simple yet powerful method that helps to remove outliers during training without severely affecting full precision accuracy and provides a quantization or compression friendly checkpoint. Using R^2 loss we intend to trim the edges of the black distribution and convert it to a distribution similar to the one shown in red in Figure 1 (left). The expected behavior of R^2 loss is that it possibly would regress full-precision model's accuracy slightly as it works as additional weight regularization like weight decay. However, the full-precision model trained with R^2 loss would have a quantization/compression friendly weight distribution removing outlier weights, so lower bit quantization accuracy can be improved as shown in Figure 1 (right). In case of higher bit quantization such as 4bit or 8bit, a

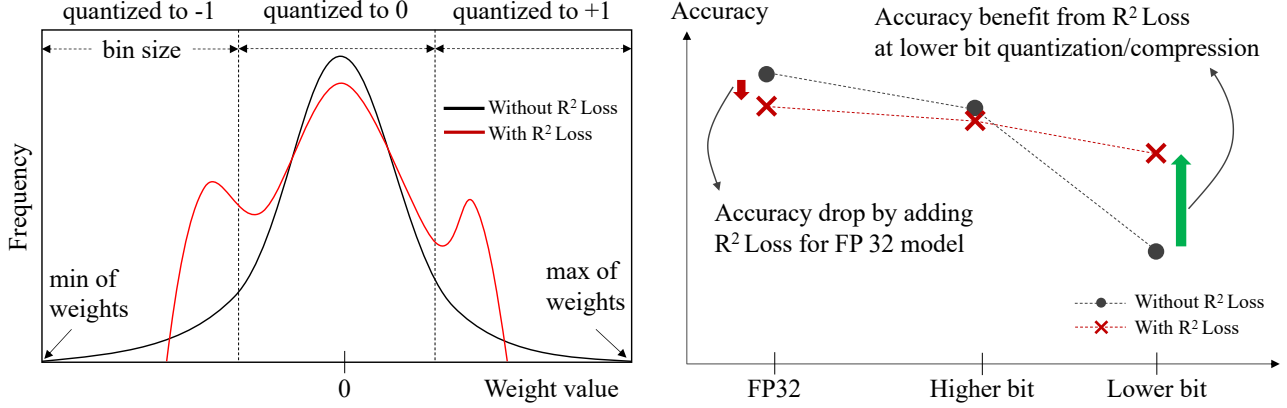


Figure 1. (Left) Example weight distribution for weight quantization with three bins. (Right) Expected benefit of R^2 loss for low bit quantization/compression.

model might already have enough bits to properly represent weights even including outliers. Therefore the benefit of R^2 loss could be limited. Also as state-of-the-art techniques show reasonable accuracy at higher bit quantization at marginal accuracy regression from full-precision models, there is not much room for improvement at higher bit quantization/compression.

We propose three different formulations of R^2 loss and through experiments show that models trained with them are more quantization and compression friendly. L_∞ R^2 loss is simple and intuitive to penalize the outlier weights during pre-training. Margin R^2 loss is an extension of L_∞ . It penalizes weights which are larger than a margin, while minimizing the width of the margin. Soft-Min-Max R^2 loss smoothly penalizes not only outliers but also near-outlier weights. These three R^2 loss can be used in different cases considering their characteristics. L_∞ and Margin R^2 loss would be more effective to symmetric quantization as they push to a symmetric weight distribution. Soft-Min-Max R^2 loss could be better for compression than others as it makes asymmetric weight distribution and smoothly regularizes weights with a differentiable parameter.

We show that R^2 loss works well with state-of-the-art quantization and compression algorithms by conducting accuracy evaluation with many different quantization and compression approaches such as DFQ(Nagel et al., 2019), AdaRound(Nagel et al., 2020), SQuant(Guo et al., 2022), PD-Quant(Liu et al., 2023), PACT(Choi et al., 2018), LSQ(Esser et al., 2019), EWGS(Lee et al., 2021), and DKM(Cho et al., 2021) with various models like MobileNet-V1, MobileNet-V2, ResNet18, ResNet50, ResNet101, and MobileBERT(Sun et al., 2020).

Our experiments on MobileNet-V1 (Howard et al., 2017) and MobileNet-V2 (Sandler et al., 2018) using post training quantization techniques like DFQ (Nagel et al., 2019) shows $> 10\%$ improvement using R^2 loss trained models.

Even with newer methods like SQuant (Guo et al., 2022) we observe an improvement of $> 5\%$. R^2 loss also shows similar gain in accuracy for PTQ methods (AdaRound, SQuant, PD-Quant (Nagel et al., 2020; Guo et al., 2022; Liu et al., 2023)) for larger models like ResNet-50 and ResNet-101 (He et al., 2016). For QAT methods like EWGS (Lee et al., 2021) R^2 trained models are roughly 4% better than the ones trained without R^2 loss for 2bit weight and activation quantization. For models compressed using 32x compression, R^2 loss improves the accuracy of parameter constrained models like MobileNet-V1 by 5% (2-bit, 2-dim). We have also extended R^2 loss to fine-tuning MobileBERT on QNLI task, where we see an absolute 2% improvement in accuracies for 1-bit and 2-bit compression.

2. Related Works

2.1. Model compression

The simplest and one of the most effective form of compression involves sharing weights within a layer. Deep Compression (Han et al., 2016) introduced k-means clustering based weight sharing for compression. Initially, all weights belonging to the same cluster, share weight of the cluster centroid. During forward pass, loss is calculated using the shared weights which are then updated during backward pass. This leads to a loss of accuracy and model train-ability because the weight to cluster assignment is intractable during weight update (Yin et al., 2019). DKM (Cho et al., 2021) introduces differentiable k-means clustering, therefore making cluster assignments tractable. During forward clustered weights are used, however during backward the gradient is applied on the original weights.

2.2. Model quantization

Model quantization reduces the memory footprint of a model by reducing the representative bits per weight for a given model. It also quantizes activation values so that we

can convert floating point computation into integer computation which gives us a benefit of hardware efficiency. In this paper we have applied our R^2 loss with various training time quantization (quantization-aware training, QAT) algorithms like EWGS (Lee et al., 2021), LSQ (Esser et al., 2019) and DoReFa (Zhou et al., 2016) used in PACT (Choi et al., 2018). PACT clips activation values with a trainable parameter for activation quantization and uses DoReFa for weight quantization. LSQ quantizes weights and activations with learnable step size (scale or bin size). EWGS applies gradient scaling with respect to position difference in between original full precision weights and quantized weights based on LSQ.

Also, we have compared our Range Restriction Loss with a state-of-the-art post-training quantization (PTQ) methods. DFQ (Nagel et al., 2019) equalizes per-channel weight ranges by applying per-channel scaling factors. It resolves the wide weight range problem across channels, but still the weight range would remain wide for lower bit quantization like 4bit as DFQ does not target outliers within a channel. In our experiment, models trained with our Range Restriction Loss can be effectively quantized to 4bit weight / 8bit activation by PTQ without DFQ. AdaRound (Nagel et al., 2020) proposed adaptive rounding for quantization bin assignment instead of nearest rounding. Pre-trained models with Range Restriction Loss also show better quantization accuracies with AdaRound than models trained with just L2 norm which is well-known regularization. SQuant (Guo et al., 2022) decomposes a layer by the Hessian-based optimization objective into three diagonal sub-items, element-wise, kernel-wise, and output channel-wise. It then compose the sub-items in a quantized domain with respect to constrained absolute some of error. PD-Quant (Liu et al., 2023) quantizes weights by comparing model prediction result before and after quantization of each layer.

In our extensible experiments, we show our Range Restriction Loss improves accuracies with cutting-edge QAT and PTQ for lower bit quantization like 2bit weight / 2bit activation and 4bit weight / 8bit activation.

2.3. Regularization for quantization

Regularization is a well-known technique for over-fitting. But some research works used the regularization concept in the context of quantization. (Shkolnik et al., 2020) show that uniform distribution of weights are more robust to quantization than normally-distributed weights. To this they propose KURE (Kurtosis REgularization) to minimize the kurtosis of weights and ensure a uniform distribution. This method is independent of the quantization bit-width, therefore supports PTQ (Post-Training Quantization) in addition to QAT (Quantization Aware Training). However, this method is best suited for generic models

which need to be specifically tuned for a given bit precision use case. To reduce the accuracy drop due to quantization (Han et al., 2021) proposes to constrain weights to predefined bins based on the quantization bit-width. However, selecting these bins is a difficult process and the output of the models in very sensitive to the bin selection. In addition to that these methods ignore the effect of quantizing the first and last layers of the models.

A key difference between R^2 loss and the existing regularization research for quantization is that other works does not explicitly consider the outliers nor weight ranges in a pre-trained model. Therefore, as shown in Figure 2, the model trained with KURE still have wider weight ranges than models trained with R^2 loss so that a quantization model from KURE would have a larger bin size which leads to bigger quantization error. Therefore, in our comparison experiment, our R^2 loss shows noticeable improvement from KURE.

3. Range Restriction Loss

We introduce Range Restriction Loss as an auxiliary loss to reduce the range of weights for every layer to get better pre-trained models for further quantization or compression. Just like L_1 and L_2 regularization our approach is invariant to the quantization or compression technique used. But as opposed to L_1 or L_2 regularization, Range Restriction Loss only affects the outliers in weights by penalizing them while maintaining accuracy of the full precision model. We demonstrate that L_2 regularization (1x(baseline) and 10x(heavy L_2)) does not solve the problem of outliers in Figure 2. As a reference for the expected weight distribution for a quantization friendly model we use the weight distribution from a model trained using KURE (Shkolnik et al., 2020). While, the idea of minimizing range can be formulated in various ways we propose 3 different ways of defining R^2 . We start from L_∞ loss, extend it to margin loss and finally introduce soft-min-max loss for adding R^2 loss to the training loss.

$L_\infty R^2$ loss: This method tries to penalize only the outliers in an iterative manner during training by adding $L_\infty(W)$ as an auxiliary loss for every layer in the model.

$$L_{reg} = \sum L_\infty(W) \quad (1)$$

The effect of this formulation is described in Figure 2 where it shows that Range Restriction Loss helps to get rid of all outliers in the model. In addition, it brings the overall range of weight down in contrast to KURE (Shkolnik et al., 2020), while, also making the weight distribution similar to a mixture of Gaussians as seen in KURE.

Margin R^2 loss: This is an extension of $L_\infty(W)$ R^2 loss, where, we define a margin for the range of allowed weights.

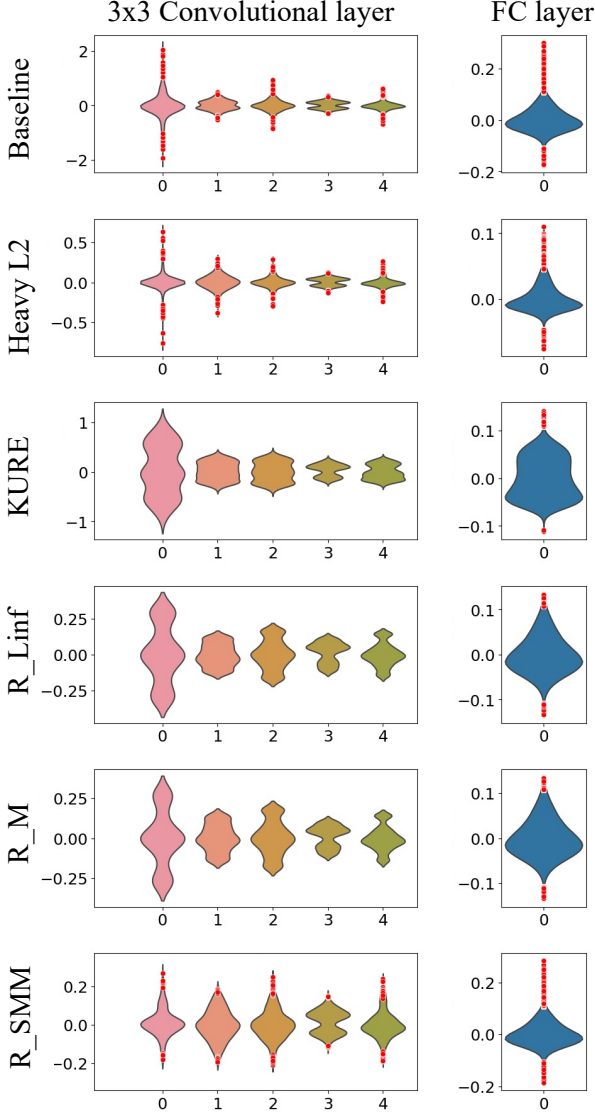


Figure 2. Weight distribution of the first five 3x3 convolutional layers and FC layer of MobileNet-V2 using L2 norm (baseline), heavy L2 norm (10x heavy L2 norm than baseline) and the proposed Range Restriction Loss (the red dots correspond outliers). KURE: Kurtosis Regularization (Shkolnik et al., 2020).

Any weight outside this margin is penalized. In addition, we also penalize the width of the margin to ensure that the range of the overall weight distribution is small. The auxiliary loss for a given weight W is shown in Equation (2). Here M is a learnable parameter per layer.

$$L_{reg} = \sum (|M| + \max(|W| - |M|, 0)) \quad (2)$$

The effect of margin R^2 loss is similar to that of L_∞ in terms of the final weight distribution as evident from Figure 2. The only difference is that margin R^2 loss penalizes all weight outside the margin per iteration in contrast to penalizing only the maximum weight.

Soft-min-max R^2 loss: In this approach, we propose an asymmetric R^2 loss, to eliminate the constraint on the magnitude of weights and strictly enforce it on the range of weights. We hypothesize that such a technique will improve asymmetrically quantized/compressed models using techniques such as DKM (Cho et al., 2021). The loss for a given weight W is described in Equation (3).

$$\begin{aligned} s_{max} &= \frac{\sum (W \odot e^{\alpha \times (W - W_{max})})}{\sum e^{\alpha \times (W - W_{max})}} \\ s_{min} &= \frac{\sum (W \odot e^{-\alpha \times (W - W_{min})})}{\sum e^{-\alpha \times (W - W_{min})}} \\ L_{reg} &= (s_{max} - s_{min}) + e^{-\alpha} \end{aligned} \quad (3)$$

Here temperature α is a learnable parameter per layer. $e^{-\alpha}$ term in the auxiliary loss L_{reg} , encourages temperature α to increase during training time optimization process to approach hard-min-max loss towards the end of training. This loss smoothly penalizes not only outliers but also near-outlier weights together rather strictly brings only outliers down like other R^2 losses. Therefore, it might be susceptible to outliers as seen in Figure 2.

All the above mentioned R^2 loss were employed during training time of the base model itself and not during quantization or compression. This was done because the purpose of R^2 loss is to provide effective initial weights for compression or quantization. This ensures extensibility of R^2 loss to any quantization or compression technique.

4. Experiment

4.1. Experiment settings

4.1.1. PRE-TRAINING FROM SCRATCH WITH AND WITHOUT R^2 LOSS

We train ResNet-18 (He et al., 2016), MobileNet-V1 (Howard et al., 2017) and MobileNet-V2 (Sandler et al., 2018) on ImageNet 1K (Deng et al., 2009) with proposed Range Restriction Loss on a x86 Linux machine with eight GPUs to get pre-trained models before model compression and quantization-aware training. We set initial learning rates to 1.0, 0.4 and 0.4 for ResNet-18, MobileNet-V1 and MobileNet-V2 respectively. We use SGD with 0.9 of momentum with Nesterov. We apply $1e-4$ of weight decay (L2 norm weight regularization) for ResNet-18 and $4e-5$ for MobileNet-V1 and V2. For heavy L2-regularization, in Figure 2, we use $4e-4$ of weight decay (10x heavier than baseline) for MobileNet-V2 to see whether heavy L2-regularization helps quantization or not as a naive solution for range restriction. Strength of R^2 loss is set to 0.01. For Margin R^2 loss, the margin threshold is initialized with 2x the standard deviation of the initialized weights. In Soft-min-max R^2 loss training, the learnable parameter α is initially set to 0.1. For comparison, we use pre-trained models

Table 1. Model size of compressed MobileNet-V1 (in M bytes). All: all layer quantization. W/O F&L: Quantize the model excluding the first and last layers. 2bit accuracy: EWGS 2bit weight and activation quantization accuracies on ImageNet. The model size of FP32 MobileNet-V1 is 16.1 MB

MOBILENET-V1	ALL	W/O F & L
4BIT	4.2	7.1
MODEL SIZE (MB)	2.2	5.6
1BIT	1.2	4.8
2BIT ACCURACY	55.96%	59.10%
2BIT ACCURACY WITH R^2	59.05%	61.25%

of Resnet-18 from Torchvision. As we are using modified version of ResNet-50, and ResNet-101, MobileNet-V1 and V2 for better FP32 performance, we trained those models from scratch without R^2 loss using the same settings above. It can be observed from Table 5 that R^2 loss does not significantly affect the performance of the full precision models as well therefore provides a strong initial checkpoint for the model to be quantized.

4.1.2. MODEL COMPRESSION AND QUANTIZATION

R^2 loss is not a model compression nor quantization method. It penalizes outlier weights during training of the base model from scratch. To evaluate the effectiveness of R^2 loss with model compression and quantization, we apply state-of-the-art compression/quantization techniques, DKM (Cho et al., 2021), LSQ (Esser et al., 2019), EWGS (Lee et al., 2021), DFQ (Nagel et al., 2019), AdaRound (Nagel et al., 2020), SQuant (Guo et al., 2022), and PD-Quant (Liu et al., 2023) to the pre-trained model with and without R^2 loss. Except EWGS¹, SQuant² and PD-Quant³, since other works do not provide official implementation, we implement those techniques ourselves and for DFQ, and AdaRound, we used AIMET⁴.

We follow the same hyper-parameters used in the works, but **we apply compression and quantization for all layers including the first and last layers**. It is important to compress/quantize all layers including first and last layers considering computation burden at the first layer with a large convolutional filter size such as 7x7 convolutions in the first layer of ResNet and the large number of weights in the last linear layer, e.g., 1.2M of weights in the last layer of MobileNet-V1 which has 4.2M of weights in total. We have demonstrated this burden in Table 1 for more clarity. Due to the outliers and wide weight ranges in the

first and last layers, quantizing all layers have less accuracy than quantizing a model excluding the first and layer layers as shown in Table 1. We represent L_∞ R^2 loss as R_Linf , Margin R^2 loss as R_M and Soft-min-max R^2 loss as R_Smm in the results.

4.2. Model quantization

4.2.1. POST-TRAINING QUANTIZATION, PTQ WITH R^2

We compare models trained using R^2 loss and other weight regularization, L2, heavy L2, and KURE (Shkolnik et al., 2020) and quantized using PTQ methods such as DFQ (Nagel et al., 2019), AdaRound (Nagel et al., 2020), SQuant (Guo et al., 2022), and PD-Quant (Liu et al., 2023). There are two major techniques in DFQ, bias correction compensating bias in activation and cross-layer equalization applying scale factor per channel to make all channels in a layer have similar weight range. AdaRound adaptively rounds weights to quantization bins instead of naive nearest rounding. SQuant decomposes a layer by the Hessian-based optimization objective into three diagonal sub-items, element-wise, kernel-wise, and output channel-wise, and then it compose the sub-items in a quantized domain with respect to constrained absolute some of error. PD-Quant quantizes weights by comparing model prediction result before and after quantization of each layer. Unlike other PTQ, SQuant dose not require calibration dataset, so its accuracy could be less than others.

As shown in Table 2, models trained with R^2 loss are more quantization friendly than other regularization. As KURE makes the weight distribution uniform, it can reduce outliers as a side-effect while keeping a wide weight range. Therefore, KURE is more effective than L2 norm, but R^2 loss shows the best accuracy as it reduces outliers as well as weight range. On the other hand, heavy L2 regularization (10x L2) makes weight ranges smaller, but it does not remove outliers, therefore prove to be ineffective here.

In comparison between FP32 accuracy of baseline models and models trained with R^2 loss, we can see there are slight accuracy regression in full-precision inference as we expected in Figure 1 (right). However, after quantizing, the models trained with R^2 loss shows better accuracies for all PTQ methods that we used in Table 2. KURE regulates entire weights to make them uniform distribution, so it is a harsh regularization. This is the reason why FP32 accuracy with KURE is inherently less than other cases including R^2 loss which only affects to outlier (and near-outlier) weights. Also, even considering FP32 accuracy difference, still R^2 loss achieves better performance in terms of accuracy regression from FP32 model to quantized model, e.g., KURE regresses by 11.99% and 1.11% by using AdaRound and PD-Quant for MobileNet-V1, respectively (72.50% $\rightarrow$ 60.51% and 71.39%), while Margin

¹<https://github.com/cvlab-yonsei/EWGS>

²<https://github.com/clevercool/SQuant>

³<https://github.com/hustvl/PD-Quant>

⁴<https://quic.github.io/aimet-pages/>

Table 2. Top-1 accuracies (%) of MobileNet-V1 and V2 on ImageNet-1K using PTQ methods with 4bit weight and 8bit activation quantization. Heavy L2: applied 10x heavy L2 regularization than baseline. KURE (Shkolnik et al., 2020). Naïve: quantizing without any advanced PTQ techniques. DFQ (Nagel et al., 2019), AR (Nagel et al., 2020), SQ (Guo et al., 2022), PD-Q (Liu et al., 2023)

METHOD	MOBILENET-V1						MOBILENET-V2					
	FP32	NAÏVE	DFQ	AR	SQ	PD-Q	FP32	NAÏVE	DFQ	AR	SQ	PD-Q
BASLINE	74.12	2.67	54.06	70.42	63.85	71.87	73.08	2.57	56.56	71.29	59.30	71.54
HEAVY L2	72.67	13.41	57.68	69.23	66.51	69.86	71.00	4.17	0.09	68.06	57.30	68.92
KURE	72.50	53.69	59.21	60.51	68.84	71.39	72.49	39.21	62.49	71.68	66.09	71.88
R_LINF	73.65	61.73	53.00	72.28	69.69	72.76	72.64	59.95	62.39	71.76	64.02	72.01
R_M	73.54	61.66	65.06	72.29	68.96	72.77	72.73	60.03	66.04	71.89	67.17	71.92
R_SMM	73.95	44.24	59.21	71.35	67.86	72.39	72.81	36.69	51.46	71.77	67.16	71.98

Table 3. Top-1 accuracies (%) of 2bit weight and 8bit activation PTQ using PD-Quant (Liu et al., 2023). MNV1: MobileNet-V1, MNV2: MobileNet-V2, RN50: ResNet-50, RN101: ResNet-101. FP32 accuracy is in Table 2 and Table 4.

METHOD	MNV1	MNV2	RN50	RN101
BASLINE	47.62	50.66	62.92	66.50
R_LINF	56.79	59.49	71.34	73.89
R_M	56.27	58.70	71.65	73.87
R_SMM	54.20	57.53	69.31	71.24

R^2 loss regresses by 1.25% and 0.77% (73.54% $\rightarrow$ 72.29% and 72.77%).

Even without advanced PTQ approaches such as DFQ, AdaRound, SQuant, and PD-Quant, models trained with R^2 loss can be reasonably quantized without any further fine-tuning. In Table 2, Naïve with R^2 loss shows significantly higher accuracy than other regularization. The models with R^2 loss have good weight distribution already from pre-training so that they can be quantized with fairly high quantization accuracies.

We conduct further studies, lower bit PTQ (2bit weight and 8bit activation) and PTQ for larger models (ResNet50 and ResNet101) as shown in Table 3 and Table 4. From the lower bit PTQ result, we can clearly see the benefit of R^2 loss. For example, accuracy of ResNet101 trained with L_∞ R^2 loss only regressed by 5.41% (79.30% $\rightarrow$ 73.89%), while the baseline model trained without R^2 loss shows huge regression, 12.95% (79.45% $\rightarrow$ 66.50%).

4.2.2. QUANTIZATION-AWARE TRAINING, QAT WITH R^2 LOSS

We apply state-of-the-art quantization techniques like PACT (Choi et al., 2018) while training the models from scratch using R^2 loss. For other quantization aware training methods like, EWGS (Lee et al., 2021) and LSQ (Esser et al., 2019) we initialize the model to pre-trained ResNet-18 (RN) (He et al., 2016), MobileNet-V1 (MN1) (Howard

Table 4. Top-1 accuracies (%) of ResNet50 and ResNet101 on ImageNet-1K with 4bit weight and 8bit activation PTQ.

METHOD	RESNET50			
	FP32	AR	SQ	PD-Q
BASLINE	78.04	74.73	74.68	76.60
R_LINF	77.97	76.34	76.28	77.50
R_M	78.11	77.31	75.32	77.78
R_SMM	78.22	75.61	75.75	77.21

METHOD	RESNET101			
	FP32	AR	SQ	PD-Q
BASLINE	79.45	75.18	75.23	78.16
R_LINF	79.30	78.22	76.87	78.61
R_M	79.27	78.19	77.95	78.69
R_SMM	79.63	76.71	77.20	78.49

et al., 2017) and MobileNet-V2 (MN2) (Sandler et al., 2018) with R^2 loss.

Table 5 shows 2 bit weight and activation quantization result of MobileNet-V1 and V2 using EWGS (Lee et al., 2021) with various regularization such as without R^2 loss (only with L2 norm), KURE (Shkolnik et al., 2020), and our R^2 loss. For both the models with varying model sizes, R^2 loss outperforms the models trained with only L2 norm or KURE. Without R^2 loss, accuracy of MobileNet-V1 2bit quantization using EWGS declines from 59.10% to 55.96% when we quantize all layers including the first and last layers as shown previously in Table 1. This is because the first and last layers have wide weight ranges and many outliers as shown in Figure 2. Our approach effectively reduces the outliers in the first and last layer which enables the 2bit quantized model to achieve similar accuracy to the case with original EWGS results where the first and last layers of the model remain in FP32.

As shown in Table 6, R^2 loss helps the quantization techniques in improving their accuracy, especially for ex-

Table 5. Top-1 accuracies (%) of MobileNet-V1 and V2 on ImageNet-1K with EWGS varying model size. Weights and activations are quantized with the same 2bit. Model sizes are adjusted by varying width factor (α in (Howard et al., 2017) and width multiplier in (Sandler et al., 2018)). Width factor for large: 1.0, medium: 0.5, and small: 0.25.

METHOD	MOBILENET-V1						MOBILENET-V2					
	LARGE		MEDIUM		SMALL		LARGE		MEDIUM		SMALL	
	FP32	2BIT	FP32	2BIT	FP32	2BIT	FP32	2BIT	FP32	2BIT	FP32	2BIT
BASELINE	74.12	55.96	66.51	38.77	55.43	20.85	73.08	53.93	65.55	39.25	53.90	27.82
KURE	72.50	57.80	63.99	39.37	52.83	22.16	72.49	53.97	64.64	38.51	52.80	26.86
R_LINF	73.65	59.05	65.68	41.22	53.48	26.04	72.64	56.35	65.09	42.55	53.40	30.56
R_M	73.54	58.41	65.83	42.61	53.30	24.35	72.73	57.36	64.86	43.78	52.82	29.61

Table 6. Top-1 accuracies (%) of ResNet18 with various QAT methods. Weights and activations are quantized with the same bit (2W2A: 2bit, 4W4A: 4bit).

METHOD	2W2A EWGS			
	FP32	PACT	LSQ	EWGS
BASELINE	69.76	51.97	58.33	65.42
R_LINF	70.15	55.26	62.23	65.72
R_M	70.08	56.24	62.25	64.27
R_SMM	69.84	55.64	62.47	64.94

METHOD	4W4A EWGS			
	FP32	PACT	LSQ	EWGS
BASELINE	69.76	66.90	69.90	70.19
R_LINF	70.15	68.45	69.55	70.17
R_M	70.08	68.30	69.56	69.77
R_SMM	69.84	68.36	69.45	69.80

tremely low bit quantization such as at 2 bit while it shows similar accuracies with 4 bit. For example, all R^2 losses improve 2 bit quantization accuracy with LSQ to over than 62% from 58%, but there is no noticeable difference in 4 bit LSQ accuracies with and without R^2 . The reason why R^2 loss would not help much for higher bit like 4 bit quantization is that QAT can effectively represent outliers using many bits as we expected in Figure 1 (right).

Interestingly, soft-min-max R^2 loss does not seem to be as good as L_∞ or margin R^2 loss for quantization. As discussed in Section 3, soft-min-max R^2 loss allows us to have an asymmetric weight distribution so that it would be more effective for model compression instead of symmetric model quantization.

4.3. Model compression

We evaluate the effectiveness of R^2 loss for compression with the state-of-the-art compression technique, DKM (Cho et al., 2021), for ResNet-18 and MobileNet-V1. The bit-dim ratio, $\frac{b}{d}$ is an important factor in the DKM algo-

Table 7. Top-1 accuracies(%) of compression using DKM with ResNet18 (RN), MobileNet-V1 (MN1) on ImageNet varying compression bit. FP32 accuracy is in Table 5 and Table 6.

METHOD	R^2	RN	MN1
DKM 1-BIT, 1-DIM	BASELINE	58.97	45.54
	R_LINF	59.52	49.74
	R_M	59.70	47.21
	R_SMM	59.27	52.58
DKM 2-BIT, 1-DIM	BASELINE	67.64	65.95
	R_LINF	68.53	67.06
	R_M	68.33	67.50
	R_SMM	68.63	67.62
DKM 4-BIT, 1-DIM	BASELINE	70.22	69.29
	R_LINF	70.34	69.43
	R_M	70.33	68.52
	R_SMM	70.52	69.63

rithm which effectively defines the kind of compression a DKM palettized model would see. We ran these experiments for both scalar and vector palettization. For scalar palettization($dim = 1$) we ran 1 bit, 2 bit and 4 bit compression. These experiments would yield roughly 32x, 16x and 8x compressed models respectively. Table 7 shows that R^2 loss significantly improves accuracy from original scalar palettized DKM 1 bit, 2 bit and 4 bit models. As we discussed, there is no significant difference for higher bit compression like 4 bit because many bit compression can also cover outliers even without R^2 loss.

We also expand the application of R^2 loss to vector palettization($dim > 1$) DKM (Cho et al., 2021) as demonstrated in Table 8. For these experiments, we kept the effective bit-dim ratio, $\frac{b}{d}$ equivalent to 1 so as to see variation across the most heavily compressed model which would be close to 32x compressed. Since a vector palettized model will require range constraining for all dimensions, we applied multi-dimensional R^2 loss for all layers that would

Table 8. Top-1 accuracies(%) of compression using multi-dimensional DKM and R^2 loss with ResNet18 (RN), MobileNet_V1 (MN1) on ImageNet.

METHOD	R^2	RN	MN1
DKM 2-BIT, 2-DIM	BASELINE	63.52	48.16
	R_SMM	64.64	53.99
DKM 4-BIT, 4-DIM	BASELINE	64.89	58.55
	R_SMM	66.10	60.05

be palettized during compression. For vector palettized ResNet-18 there is an average absolute improvement of $> 1\%$ using models trained with R^2 loss, and for vector palettized MobileNet-V1, the gain ranges from 5% to 3%.

Finally we also validated that R^2 loss scales to other domains as well by applying it in compressing MobileBERT (Sun et al., 2020). For Question Answering (QNLI) (Rajpurkar et al., 2016) using MobileBERT, R^2 loss improved the performance of the model by 2% absolute as demonstrated in Table 9. Note that we applied R^2 loss to a QNLI fine-tuning task based on a pre-trained MobileBERT (Wolf et al., 2020). It might be necessary to apply R^2 to the entire training task of MobileBERT from scratch so that R^2 loss would have more chances to get effective weight distribution for model compression. Through these experiments across a variety of tasks(Image Classification, Question Answering etc.) we can also see that the application of the proposed R^2 loss is task-invariant and yields solid results across domains.

Table 9. Question-answering NLI (QNLI) accuracies of MobileBERT using single dimension DKM

METHOD	PRE-TRAIN	1-BIT	2-BIT
DKM BASELINE	90.41	61.34	80.12
DKM + R_LINF	90.66	63.17	82.13
DKM + R_M	89.09	61.80	80.98
DKM + R_SMM	90.83	61.49	80.87

4.4. Strength of R^2 loss & comparison between R^2 loss

In this paper, we propose three different R^2 loss, L_∞ , Margin, and Soft-Min-Max. As we discussed in Section 3 and Section 4.2.2, Soft-Min-Max R^2 loss would be more effective for model compression as it allows to have asymmetric weight distribution while other R^2 loss look better for symmetric quantization. To compare L_∞ and Margin R^2 loss and see impact of strength of R^2 loss, we conduct a further ablation study varying strength of R^2 loss as shown in Figure 3. While L_∞ R^2 loss shows somewhat fluctuated

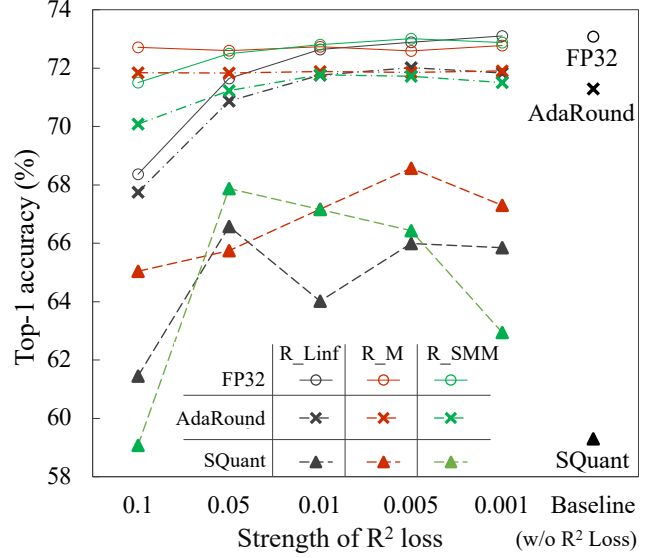


Figure 3. MobileNet-V2 result varying strength of R^2 loss. 4bit weight and 8bit activation PTQ with AdaRound and SQuant.

quantization performance with respect to its strength, quantization accuracy from a model trained with Margin R^2 loss is more consistent and better than L_∞ . L_∞ R^2 loss is naive and simple to limit the weight range. Margin R^2 loss is advanced loss from L_∞ as it applies learnable margin threshold. We think that the learnable margin parameter makes more stable full-precision pre-training while L_∞ always penalizes only one outlier per iteration. Therefore, Margin (as well as Soft-min-max) R^2 loss shows more stable full-precision accuracy before quantization. We would recommend to use Margin R^2 loss for symmetric quantization if it is hard to find proper strength for L_∞ R^2 loss.

5. Conclusion

In this paper, we introduced Range Restriction Loss as an effective technique to get rid of outliers in the weight distribution during training. This serves as a good initialization for state of the art post training quantization, quantization aware training and compression strategies, therefore can be coupled with any of them. This helps to augment the accuracy gained from such techniques and is invariant to the quantization or compression algorithm. With the three proposed formulations of R^2 loss we wanted to demonstrate that there can multiple approaches of defining R^2 loss and the final metric depends on these formulations. We also demonstrated how the proposed method converts a wide weight range distribution to a more densely-packed distribution for model quantization. While full-precision accuracy with R^2 loss can be slightly regressed as it penalizes outlier weights, R^2 loss significantly improves quantization and compression accuracy, especially for lower bit.

6. Impact Statement

Deep learning models play a pivotal role in multiple applications today. With time these models have become larger and larger in search of higher accuracy. However, deploying such models on edge devices is challenging because of their size. We introduce a simple yet effective algorithm to train models suitable for ultra low bit quantization and compression (e.g., 1 and 2 bit). We hope that the community will adopt this method to create compute-efficient models, resulting in reduced deployment costs. There are many potential societal consequences of our work, none of which we feel must be specifically highlighted here.

References

- Bengio, Y., Léonard, N., and Courville, A. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.
- Cho, M., Alizadeh-Vahid, K., Adya, S., and Rastegari, M. Dkm: Differentiable k-means clustering layer for neural network compression. In *International Conference on Learning Representations*, 2021.
- Choi, J., Wang, Z., Venkataramani, S., Chuang, P. I.-J., Srinivasan, V., and Gopalakrishnan, K. Pact: Parameterized clipping activation for quantized neural networks. *arXiv preprint arXiv:1805.06085*, 2018.
- Deng, J., Dong, W., Socher, R., Li, L.-J., Li, K., and Fei-Fei, L. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pp. 248–255. Ieee, 2009.
- Dong, Z., Yao, Z., Arfeen, D., Gholami, A., Mahoney, M. W., and Keutzer, K. Hawq-v2: Hessian aware trace-weighted quantization of neural networks. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS’20*.
- Esser, S. K., McKinstry, J. L., Bablani, D., Appuswamy, R., and Modha, D. S. Learned step size quantization. In *International Conference on Learning Representations*, 2019.
- Guo, C., Qiu, Y., Leng, J., Gao, X., Zhang, C., Liu, Y., Yang, F., Zhu, Y., and Guo, M. SQuant: On-the-fly data-free quantization via diagonal hessian approximation. In *International Conference on Learning Representations*, 2022. URL <https://openreview.net/forum?id=JXhROKNZzOc>.
- Han, S., Mao, H., and Dally, W. J. Deep compression: Compressing deep neural network with pruning, trained quantization and huffman coding. In Bengio, Y. and LeCun, Y. (eds.), *4th International Conference on Learning Representations, ICLR 2016, San Juan, Puerto Rico, May 2-4, 2016, Conference Track Proceedings*, 2016.
- Han, T., Li, D., Liu, J., Tian, L., and Shan, Y. Improving low-precision network quantization via bin regularization. In *2021 IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 5241–5250, Los Alamitos, CA, USA, oct 2021. IEEE Computer Society.
- He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016.
- Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., and Adam, H. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *arXiv preprint arXiv:1704.04861*, 2017.
- Langley, P. Crafting papers on machine learning. In Langley, P. (ed.), *Proceedings of the 17th International Conference on Machine Learning (ICML 2000)*, pp. 1207–1216, Stanford, CA, 2000. Morgan Kaufmann.
- Lee, J., Kim, D., and Ham, B. Network quantization with element-wise gradient scaling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6448–6457, 2021.
- Liu, J., Niu, L., Yuan, Z., Yang, D., Wang, X., and Liu, W. Pd-quant: Post-training quantization based on prediction difference metric. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 24427–24437, 2023.
- Nagel, M., Baalen, M. v., Blankevoort, T., and Welling, M. Data-free quantization through weight equalization and bias correction. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1325–1334, 2019.
- Nagel, M., Amjad, R. A., Van Baalen, M., Louizos, C., and Blankevoort, T. Up or down? adaptive rounding for post-training quantization. In *International Conference on Machine Learning*, pp. 7197–7206. PMLR, 2020.
- Rajpurkar, P., Zhang, J., Lopyrev, K., and Liang, P. Squad: 100,000+ questions for machine comprehension of text. *arXiv preprint arXiv:1606.05250*, 2016.
- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., and Chen, L.-C. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 4510–4520, 2018.

- Shkolnik, M., Chmiel, B., Banner, R., Shomron, G., Nahshan, Y., Bronstein, A., and Weiser, U. Robust quantization: One model to rule them all. In *Proceedings of the 34th International Conference on Neural Information Processing Systems, NIPS'20*, Red Hook, NY, USA, 2020. Curran Associates Inc. ISBN 9781713829546.
- Sun, Z., Yu, H., Song, X., Liu, R., Yang, Y., and Zhou, D. Mobilebert: a compact task-agnostic bert for resource-limited devices. *arXiv preprint arXiv:2004.02984*, 2020.
- Wang, K., Liu, Z., Lin, Y., Lin, J., and Han, S. Haq: Hardware-aware automated quantization with mixed precision. pp. 8604–8612, 06 2019. doi: 10.1109/CVPR.2019.00881.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., et al. Transformers: State-of-the-art natural language processing. In *Proceedings of the 2020 conference on empirical methods in natural language processing: system demonstrations*, pp. 38–45, 2020.
- Yin, P., Lyu, J., Zhang, S., Osher, S. J., Qi, Y., and Xin, J. Understanding straight-through estimator in training activation quantized neural nets. In *7th International Conference on Learning Representations, ICLR 2019, New Orleans, LA, USA, May 6-9, 2019*. OpenReview.net, 2019.
- Zhou, S., Wu, Y., Ni, Z., Zhou, X., Wen, H., and Zou, Y. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *arXiv preprint arXiv:1606.06160*, 2016.